Decide, Don't Generate



Jev, System One Models, and the Decision Layer of Agentic AI

Runnable labs · 130+ figures · no API key needed

Sridhar Mukkandi

Decide, Don’t Generate

Jev, System One Models, and the Decision Layer of Agentic AI

Sridhar Mukkandi

First edition

Decide, Don’t Generate: Jev, System One Models, and the Decision Layer of Agentic AI

Copyright © 2026 Sridhar Mukkandi. All rights reserved.

No part of this book may be reproduced, stored or transmitted in any form without written permission from the author, except for brief quotations in reviews and articles. The code in this book is released under the MIT licence at github.com/Mukkandi-Sridhar/decide-dont-generate.

Independently published. First edition, 2026.

Jev is a product of TypeSafe AI. This book is independent: the author is not affiliated with TypeSafe AI, and the book is not endorsed or sponsored by it. All trademarks belong to their respective owners.

About the numbers in this book. The author did not have access to the Jev API while writing. Every Jev response, chart and table in this book was produced by `jev-mock-synthetic`, a deterministic stand-in that copies Jev’s public interface but not its behaviour. Such material is labelled “Synthetic — not measured on real Jev”. Claims made by the vendor are labelled “vendor-reported” and cited. The security data is synthetic too: Kestrel Logistics, Harbor Pharma and everyone in them are invented.

The information in this book is provided without warranty. The author assumes no responsibility for errors or omissions, or for damages resulting from its use. Nothing in this book is legal, security-compliance or medical advice.

Cover design and typesetting by the author. Text set in Source Serif 4, Inter and JetBrains Mono, all under the SIL Open Font License.

Contact: sridhar.authorhub@gmail.com

Table of contents

Preface vii

How to read this book ix

Prologue: the night shift xiii

I Prerequisites: probability and decisions xv

1 What “learning” means 1

2 Probability, and how a machine learns it 13

3 Calibration: when 0.8 really means 80% 24

4 From probabilities to actions 35

II Prerequisites: networks, language models and agents 45

5 Deep learning in one chapter 47

6 How an LLM writes, and structured outputs 56

7 RAG, agents, and where they break 66

III Jev in depth 75

8 System 1 and System 2 77

9 Inside Jev: what we know and what we don’t 85

10 The type system: choice, score, noul 95

11 Testing Jev’s calibration yourself 104

12 The Jevons paradox of decisions 113

IV The decision layer 123

13 The bake-off: six ways to make a decision 125

14 Act, review, or escalate 136

15 A catalogue of decision patterns 152

V Building 165

16 First calls, and the mock that makes them free 167

17 A hybrid agent: Jev decides, the LLM reasons 176

18 Case study: SOC alert triage 185

19 An applications gallery 194

20 Build your own System One model 203

21 Capstone: a production decision service 212

VI What changes now 223

22 What changes now 225

Appendices 233

Key ideas 233

The Python you'll see 236

Glossary 240

Cheat sheets 244

Bibliography 247

Index 251

About the Author 253

Preface

This is a book about deciding.

It isn't about artificial intelligence in general, though you'll learn a lot of that along the way. The subject is one narrow, very common job: looking at a situation and choosing what to do about it. Is this email phishing? Which team should get this ticket? Is this login safe? Should this agent call that tool?

Software makes millions of those decisions every day. For the last few years, we've been giving more and more of them to large language models. LLMs are extremely good at reading and writing. But a decision is different from a paragraph. It's a choice between a few options, made without being sure, with a cost for getting it wrong. To make it well, you need a calibrated probability, one that means what it says, and a threshold set by what mistakes cost. And you need to know when to give the case to a person.

In 2026, a company called TypeSafe AI released Jev. It describes Jev as the first of a new kind of model, a **System One model**. These models don't write at all. They only answer typed questions with probabilities, in a single fast pass (TypeSafe AI 2026a). You'll learn to test for yourself whether Jev does what its makers claim. But the idea behind it is worth understanding whatever happens to any one product. Some decisions belong in a separate, fast, measurable layer of an AI system, and that layer needs different skills from the rest.

Who this book is for

It's for anyone who builds, buys or runs systems that make decisions. If you're new to machine learning, Part I starts from the beginning. It assumes only that you're comfortable with a little arithmetic and a little Python. If you already build with LLMs, you'll move quickly through Part II and find the main content in Parts III to V. If you lead a team, "How to read this book" suggests a shorter route.

How it's built

Every chapter follows one made-up company, Kestrel Logistics, and its security operations centre (SOC). There, a small team triages hundreds of

alerts a day: it sorts them by how likely they are to be real and how urgent they are. SOC triage turned out to be an almost perfect teaching example. It has lots of decisions, a real cost for mistakes, and a queue of people with limited time.

Every figure is drawn by code, and almost every number in the book is computed by it. Each chapter has a lab: a notebook in the book's repository that recreates its numbers and lets you change them.

Three rules I followed

No invented measurements. I didn't have access to Jev's API while writing. So every Jev number in this book comes from a mock model called `jev-mock-synthetic`. It's built to copy the real API's shape, through TypeSafe's official Python library. Those numbers are marked *synthetic*, every time. They show you how to reason and test; they are not claims about the real model.

Vendor claims are labelled. When I report what TypeSafe says about Jev, such as its speed or its price, I say it's vendor-reported and cite where it came from.

Claims go in piles. Everything said about Jev is sorted into three piles: what I could check, what the vendor reports, and what nobody outside the company knows yet. Chapter 9 explains how. I'd encourage you to keep the same three piles for anything you read about new models, including this book.

How to read this book

You don't have to read it front to back. Here are three routes, depending on where you're starting.

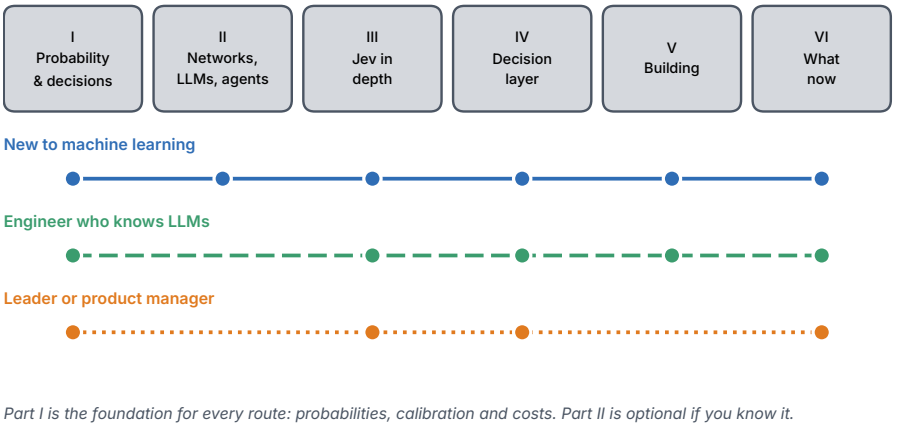


Figure 1. Three routes through the book's six parts.

If you're new to machine learning, read it in order. Part I builds the foundation for everything else: probabilities, and how to turn them into actions you can explain and defend. Part II is a short tour of neural networks, LLMs and agents. It assumes you haven't met any of them before.

If you already build with LLMs, read Part I anyway, and don't skip Chapter 3 on calibration or Chapter 4 on costs. Then go to Part III. The rest of the book depends on those two chapters more than on anything else.

If you lead a team or a product, read Part I, then Chapters 8 to 14, then Chapters 18 and 22. You'll skip the chapters with the most code and still be able to ask your team the right questions. What does a mistake cost? Where's the threshold? How do we know the probabilities are calibrated? Who reads the warnings?

The boxes

Each chapter opens with a short list of what you'll be able to do by the end. It closes with a short summary, a *Keep these* box that lists the chapter's key ideas, exercises and a link to the next chapter. In between, a few kinds of box appear again and again (Figure 2).

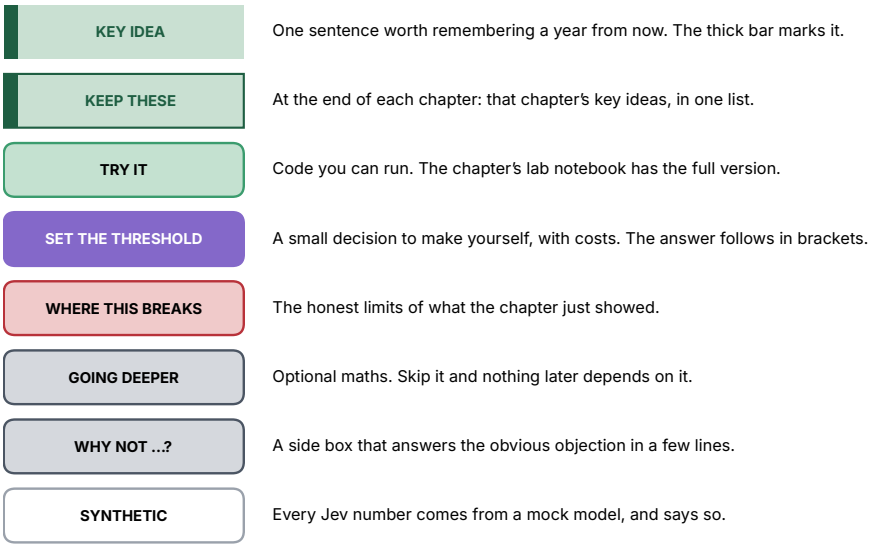


Figure 2. The boxes you'll meet in every chapter.

A *Key idea* box, with a thick bar down its left side, holds one sentence worth remembering a year from now. The *Key ideas* appendix lists all of them, with their pages, for revision.

The *Try it* boxes contain code you can run. The *Set the threshold* boxes ask you to make a small decision with real costs. The *Where this breaks* boxes show the limits of what the chapter just taught. *Going deeper* boxes hold optional maths. A side box with a question as its title, like *Why not just maximise accuracy?*, answers the objection you're probably already thinking of. Read the *Where this breaks* boxes most of all. They're the difference between knowing a method and knowing when to trust it.

Running the code

Every code listing in the book runs, and a test in the book's repository checks that it still does. The labs are Jupyter notebooks. Chapter 1's lab is `labs/ch01.ipynb`, Chapter 2's is `labs/ch02.ipynb`, and so on. Each one installs everything it needs the first time you run it.

To run things on your own machine:

```
pip install typesafe-sdk==0.7.2
pip install git+https://github.com/Mukkandi-Sridhar/decide-dont-generate
```

Prologue: the night shift

It's 2:14 a.m. at Kestrel Logistics, and you're the only analyst on shift.

The building is quiet. On your left screen, the alert queue is moving the way it always does at this hour: a sign-in from an unusual country, a program that ran from a temporary folder, an email with a link to a website created last week, a laptop talking to a server nobody recognises.

Most of them are nothing. You know that. A salesperson on a trip, an IT tool doing its job, a newsletter with a tracking link. After a few months of night shifts, you can feel which ones are nothing before you finish reading them.

But not always. And *not always* is the whole job.

You open the next alert. Twelve minutes later, you've read the logs, checked the user's history, looked up the website and decided: fine. Close it. Next.

At five alerts an hour, you'll get through about thirty-five tonight. The queue will get about two hundred.

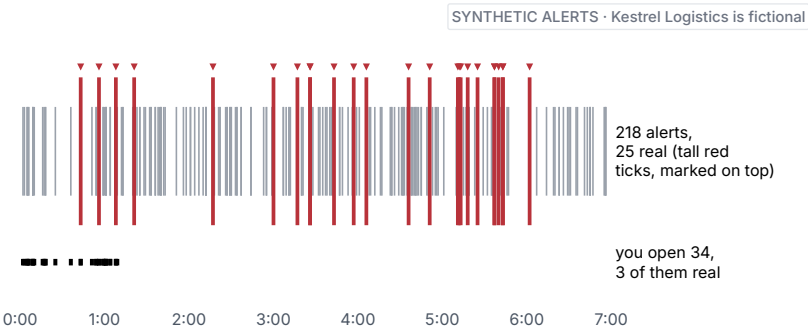


Figure 3. One night at Kestrel, midnight to 7 a.m. Each line is an alert; the real ones are red, taller and marked on top. Working in the order they arrive, you open the ones on the left and the rest wait for the morning.

Figure 3 shows the whole night at once, a view you never get from your desk. There were 218 alerts between midnight and seven. 25 of them were real. Working through the queue in order, you opened 34.

3 of those were real.

You didn't do anything wrong. You were careful and quick, and you got every one you opened right. The problem is that the queue decided what you'd look at, and the queue doesn't know anything. It only knows what arrived first.

Now imagine a different night.

A model reads every alert that arrives in a fraction of a second. It doesn't write a report or explain itself. It answers three small questions: how likely is this to be real, what kind of threat would it be, and how bad? Its answers are probabilities. Someone checked last week that they're calibrated: of the alerts it calls one in ten, about one in ten turn out to be real.

The alerts it's nearly certain are nothing get closed, and a random few of those go into a pile for you to check. For the ones it's fairly sure are serious, the analyst on call is woken up straight away. Everything in between comes to you, likeliest first.

You still make the decisions that matter. You just make them on the right alerts.

That second night is what this book builds, piece by piece. It takes surprisingly little new technology and quite a lot of careful thinking. You need to know what a probability is and how to tell whether one can be trusted. You need to know how much a mistake costs, and where to set a threshold so that the mistakes you make are the cheap ones. And you need to know what to do when the world changes, as it will, on some ordinary day.

We'll start at the very beginning, with the simplest question of all.

Next: before we can teach a machine to decide, what does it actually mean for a machine to learn?

CHAPTER 3

Calibration: when 0.8 really means 80%

When you can measure what you are speaking about, and express it in numbers, you know something about it.

— Lord Kelvin, lecture on electrical units of measurement, 1883

Here's a question that sounds silly until it costs you money:

"If a model says 80%, does that mean 80%?"

Your first answer is probably "of course": it's a probability, and that's what the number is for.

But a number is just a number. A model can say 0.8 about alerts that turn out to be attacks half the time, or nineteen times out of twenty. Nothing in the maths of the model forces its 0.8 to match reality. It matches only if we built it well, trained it on the right data and checked it. Even then, it can slowly go wrong over time.

This chapter is about checking, the most important habit in the book. Every threshold in Chapter 4, every policy in Chapter 14 and every claim about Jev in Part III depends on it.

BY THE END OF THIS CHAPTER

- Say what it means for a model to be *calibrated*, and why that's different from being accurate.
- Draw and read a reliability diagram, and compute the expected calibration error (ECE).
- Recognise the everyday mistakes that break calibration, including rebalancing your training data.
- Fix a miscalibrated model with Platt scaling, temperature scaling or isotonic regression, and know which to use.
- Check calibration within the groups you'll actually act on.

The weather forecaster's secret

Let's go back to the weather.

People love to complain about weather forecasters. But in the 1970s, researchers checked American forecasters' chance-of-rain forecasts against what actually happened. They found something surprising. When the forecasters said "30% chance of rain", it rained on about 30% of those days. When they said 70%, it rained about 70% of the time. Across the whole range, what they said and what happened matched almost perfectly (Murphy and Winkler 1977).

Compare that with most people. Ask someone for a range they're "90% sure" contains the answer to a quiz question. The true answer falls inside it far less often than 90% of the time. As a rule, people are too sure of themselves.

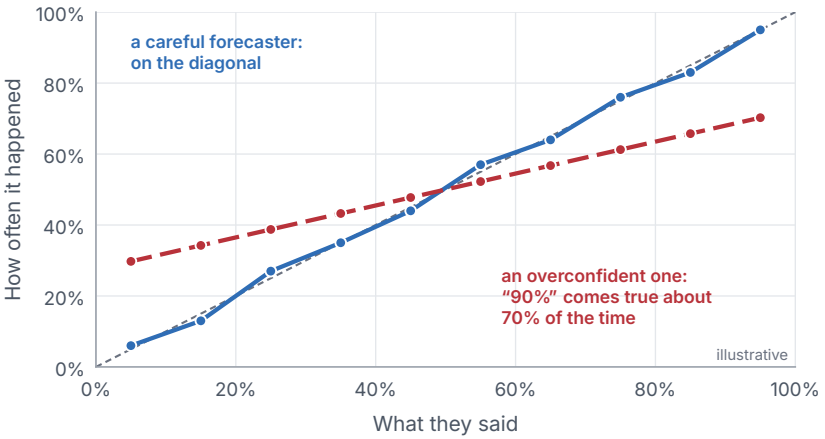


Figure 3.1. Two forecasters. For each thing they said ("30%", "70%" and so on), we count how often it actually happened. The careful forecaster sits on the diagonal: their 70% happens about 70% of the time. The overconfident one says 90% about things that happen only about 70% of the time.

The weather forecasters aren't good because they're always right. They can't be. They're good because their numbers mean what they say. That property has a name: **calibration**.

● KEY IDEA

A model is calibrated when its 80% happens about 80% of the time.

And, importantly, you can *check* calibration with nothing more than grouping and counting. Collect everything the model said "about 80%" to, and count how often it happened. A calibrated model's probabilities are honest: they mean what they say.

Good at ranking, badly calibrated

Before we check anything, you need one idea, because people confuse it all the time.

There are two separate things you might want from a model's probabilities.

The first is **ranking**: does the model give attacks higher numbers than harmless alerts? If you sort alerts by the model's score, do the real attacks rise to the top? The **AUC** measures this. It's the chance that a real attack, picked at random, scores higher than a harmless alert picked at random. An AUC of 1 means every attack scored above every harmless alert. An AUC of 0.5 means the scores are no better than tossing a coin.

The second is **calibration**: when the model says 0.3, do three in ten of those alerts turn out to be attacks?

The two are separate. A model can rank perfectly and still be wrong about how sure it is.

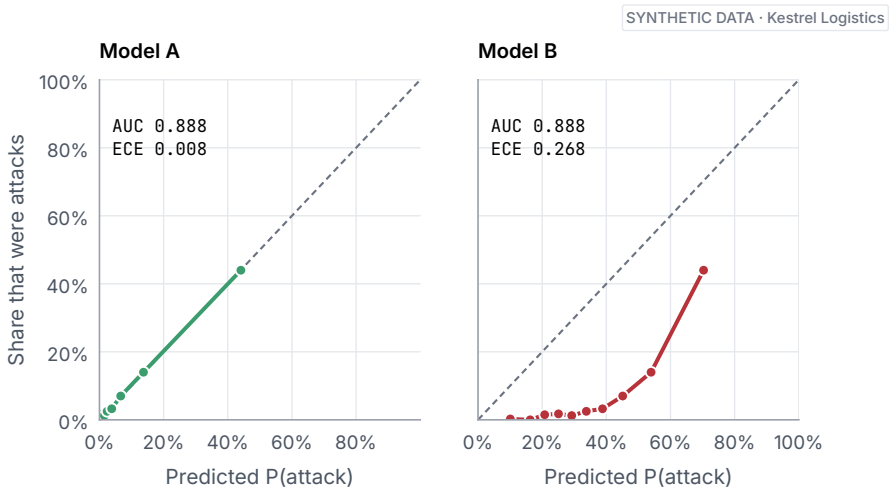


Figure 3.2. Two models, identical at ranking (same AUC) but not at calibration. Model A is the logistic regression from Chapter 2. Model B is the same model with its probabilities distorted: it still sorts the alerts in the same order, but its numbers no longer mean anything.

Figure 3.2 shows two models with the same AUC, 0.888. Model B's numbers are badly wrong: it says 40% about alerts that are attacks only a few percent of the time. If you only ever sort alerts, Model B is as good as Model A. But as soon as you put a threshold on its numbers, use them in a cost, or show them to an analyst, it'll mislead you.

This is the trap: most leaderboards and model comparisons report ranking measures like AUC, accuracy or F1. (F1 is one score that balances two

things: how many of the cases a model flags are real, and how many of the real cases it flags.) Calibration is often not measured at all. But for a decision system, it's usually what matters most.

The reliability diagram

So how do we check? The same way the weather researchers did.

1. Take a set of predictions where you know what happened: our test alerts from Chapter 2.
2. Sort them into bins by what the model said: 0–10%, 10–20% and so on.
3. In each bin, compare two numbers: the model's *average* prediction and the share of alerts that really were attacks.
4. Plot one against the other.

That plot is called a **reliability diagram**, and you'll see a lot of them from here on.

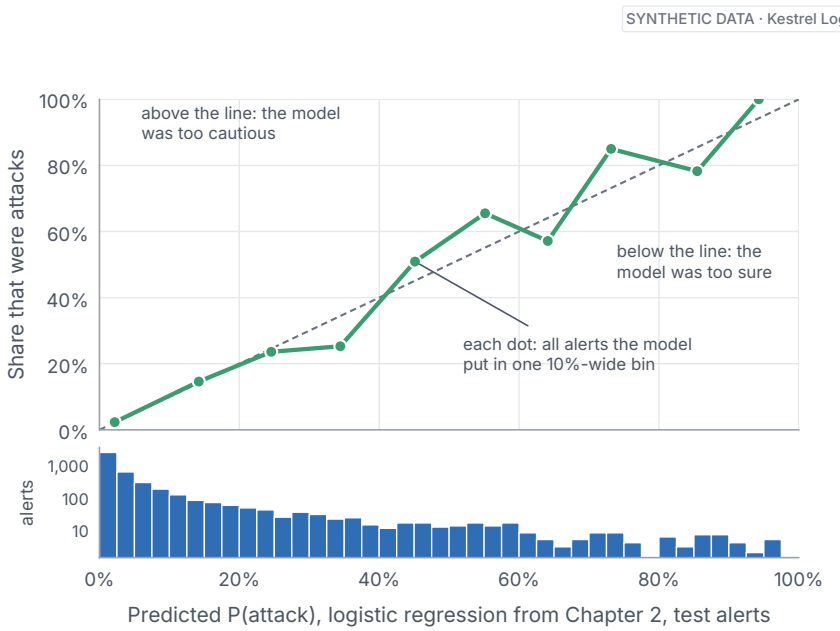


Figure 3.3. The reliability diagram of our Chapter 2 model on alerts it never saw. Points on the dashed diagonal mean the model's numbers can be trusted as they are. Underneath, a histogram (log scale) shows how many alerts fall in each part of the range: most are near zero, and only a few are above 50%.

Look at Figure 3.3 carefully, top and bottom together, because people forget the bottom panel. Our model's points are near the diagonal where there's lots of data. At the right end, the dots jump around, and the histogram tells you why: only a few dozen alerts are there. A dot based on

twelve alerts is weak evidence, as Chapter 2 warned. Always look at the counts before you worry, or before you relax.

TRY IT

Build the diagram's table yourself: bin, average prediction, observed rate, count.

```
alerts = soc.load()
train, calib, test = soc.split(alerts)
F = lambda d: soc.feature_matrix(d).to_numpy()
model = LogisticRegression(C=1e4, max_iter=5000).fit(
    F(train), train.malicious)
p = model.predict_proba(F(test))[:, 1]
y = test.malicious.to_numpy()

bins = np.clip((p * 10).astype(int), 0, 9)
table = (pd.DataFrame({"bin": bins, "said": p, "happened": y})
         .groupby("bin")
         .agg(said=("said", "mean"), happened=("happened", "mean"),
              n=("said", "size")))
print(table.round(3).head(6))
```

```
      said  happened     n
bin
0    0.022     0.023  3266
1    0.142     0.146   315
2    0.246     0.236   144
3    0.344     0.253    87
4    0.451     0.509    55
5    0.551     0.655    58
```

One number: ECE

Pictures are the best way to *see* calibration. But sometimes you need a single number, to compare models or to raise an alarm when calibration gets worse. The most common is the **expected calibration error**, or ECE. It's the average gap between "said" and "happened" across the bins, weighted by how many predictions each bin holds (Naeini et al. 2015).

```
print(f"ECE: {cal.ece(p, y):.4f}")
```

```
ECE: 0.0077
```

Our logistic regression's ECE is 0.008. That means its probabilities are off by less than one percentage point on average. Good. Model B's is 0.27.

ECE has weaknesses worth knowing. It depends on how you choose the bins. And because it's an average, it can hide very bad calibration in a small region. The high end, where the dangerous alerts are, can be badly calibrated while the rest of the data is fine. Report ECE, but always look at the picture too.

> **GOING DEEPER** optional
>
> The **Brier score** is the average squared gap between a predicted probability and what happened (1 or 0). Allan Murphy showed that it splits into three parts (Murphy 1973): Brier = *reliability* − *resolution* + *uncertainty*. **Uncertainty** is how hard the problem is to begin with (it depends only on the base rate). **Resolution** is how well the model's predictions separate cases that turn out differently; it's close to ranking skill. **Reliability** is the calibration error: the squared gap between said and happened, bin by bin. For our model: reliability 0.0005, resolution 0.0212, uncertainty 0.0698. A model can improve its Brier score by ranking better or by being better calibrated. The three parts tell you which.

How calibration breaks

If log loss rewards calibrated probabilities, as Chapter 2 said, why would a model ever come out miscalibrated? Because the way it's trained can mislead it. Here are the usual causes.

Rebalancing the data. This is one of the most common causes, and people usually do it with good intentions. Attacks are rare, only 8% of alerts. So someone decides the model needs to "see more attacks" and trains it on a 50/50 mix: every attack, plus the same number of harmless alerts chosen at random. The ranking is still fine. But the model has learned from a world where attacks are *common*, and it correctly reports probabilities for that world, not ours.

The rebalanced model's average prediction is 29%. The real attack rate is 8%. Put a threshold on those numbers and you'll send your analysts far more alerts than they can handle.

Overfitting. Flexible models that memorise, like the deep decision tree from Chapter 2, push their probabilities towards 0 and 1. On the training data, they really were that sure. They come out overconfident on new data.

Scores that were never probabilities. Some models output a "score" that happens to be between 0 and 1 but was never trained with a proper scoring rule. Treat it as a ranking until you've checked it.

CHAPTER 17

A hybrid agent: Jev decides, the LLM reasons

The greatest improvement in the productive powers of labour ... seem to have been the effects of the division of labour.

— Adam Smith, The Wealth of Nations, 1776

Chapter 7 built an agent for Kestrel. It read an alert, chose which evidence to collect, decided whether it had enough, reached a verdict and acted. We counted its steps, and more than half were small decisions.

This chapter builds that agent again, properly, as a **hybrid**: Jev makes every decision, and the LLM writes only what a person will read. Then we compare it with a version where the LLM does everything.

But first I have to tell you about a bug. It was in Chapter 7's agent from the start. It never caused a crash, and I only found it because of a habit this chapter will try to teach you.

BY THE END OF THIS CHAPTER

- Build an agent where typed decisions drive the loop and an LLM writes only for people.
- Read an agent's trace step by step, as a timeline and as an audit log.
- Find a decision step that's wrong without any error, by counting how often it gives each answer.
- Compare a hybrid agent with an all-LLM one on time, cost, review load and threats missed.
- Know which work to leave as a decision, and which to just do.

The shape of the hybrid

Figure 17.1 shows the design. Everything in green, with "Jev" in its label, is a typed question to Jev, answered in one pass. Everything in blue is a tool that reads something: the threat-intel lookup and the box labelled "tools".

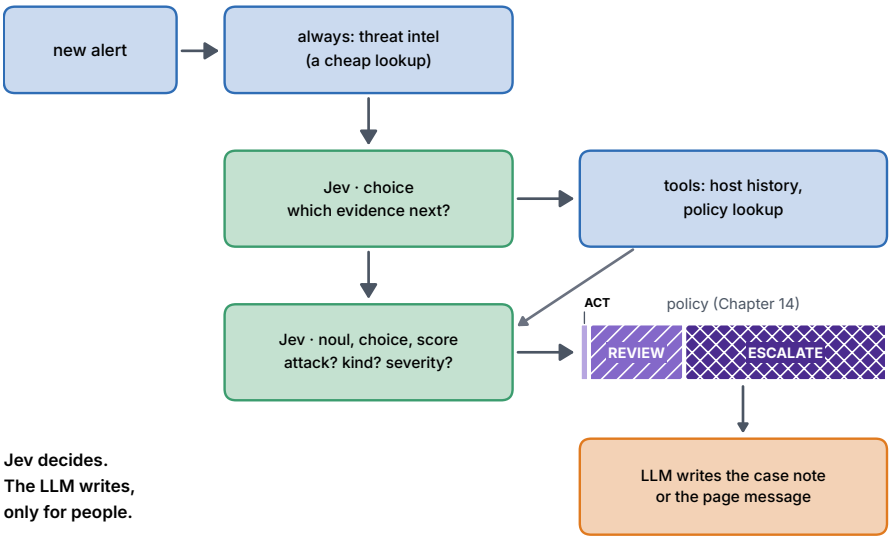


Figure 17.1. The hybrid agent. A cheap lookup always runs first. Jev decides which other evidence to collect, then answers the three verdict questions together. Chapter 14's policy turns the verdict into an action, and the LLM writes a case note or urgent message only when a person will read it.

The policy is ordinary code. The LLM, in orange and labelled "LLM", appears once, at the end. And it appears only on the paths where a person is waiting to read something.

Follow one alert through it. Each line is a step: its kind, its name, how long it took, and what it found or decided.

TRY IT

```
from textwrap import shorten
hybrid = agent.SOCAgent(always=("threat_intel",))
t = hybrid.run(live.iloc[118])
for s in t.steps:
    detail = shorten(s.detail, 34, placeholder='…')
    print(f"{s.kind:>8} {s.name:<15} {s.latency_s:4.2f}s {detail}")
print("action:", t.action)
```

```
 observe get_alert       0.05s Impossible travel sign-in
 observe threat_intel    0.30s score 0.02
  decide next_evidence   0.14s -> none (p=0.56)
  decide verdict         0.16s P(attack) = 0.000, benign,…
     act close_alert     0.05s auto-closed
action: act
```

That printout is the agent's **trace**: the step-by-step record of what it did. It's more useful than it looks. It's a debugging tool, an audit log and an explanation all at once. When someone asks next month why alert 118 was closed, you read the trace. It shows which evidence was collected, what each decision said, with what probability, and which policy threshold it crossed.

The bug that didn't crash

Notice the first line of the setup: `always=("threat_intel",)`. That wasn't in Chapter 7's version, and there's a reason it's there now.

When I first ran the agent over the live week, the results looked reasonable. Actions were spread across the three zones, every answer parsed, and nothing raised an error. Then I ran a check that's worth running on every decision step of every agent: count its answers.

Across 700 alerts, the "which evidence next?" decision chose host history, the policy lookup and "none" about equally often. It never chose threat intel. Not once (Figure 17.2).

So the verdict was always reached without the most useful fact about an alert, and the model used a default value instead. The ranking got worse: AUC 0.723, against the 0.887 it reaches with threat intel. And about 22.6 real threats a day were auto-closed.

I want to be careful about what this shows. The mock's evidence choices come from a simple word-matching engine, so I can't tell you real Jev

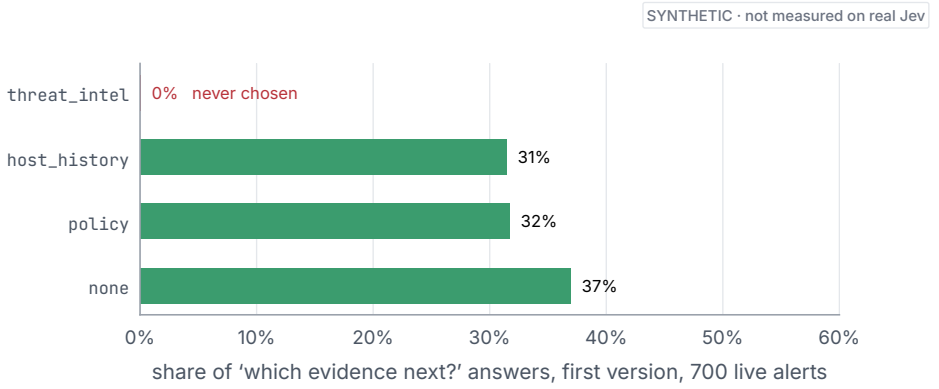


Figure 17.2. How often the first version of the agent chose each option at its "which evidence next?" step, across 700 live alerts. It never once chose threat intel.

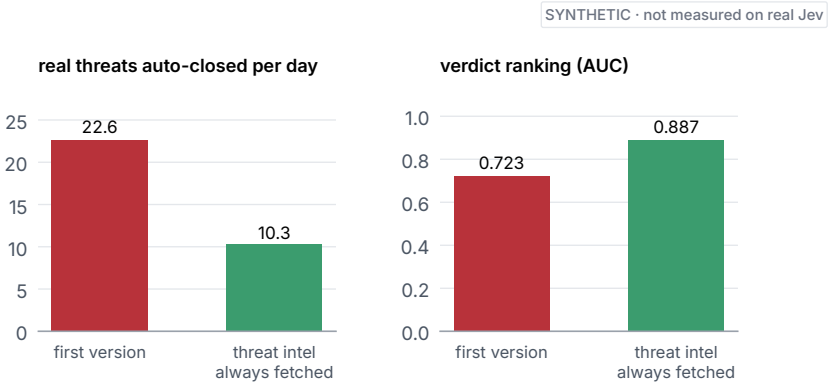


Figure 17.3. The same agent before and after one change: always fetching threat intel. Real threats auto-closed per day fell by more than half, and the verdict's ranking recovered.

would make the same mistake. What I can tell you is that *any* decision step can fail this way, whichever model answers it. And nothing later in the chain will report a problem. Typed answers always have the right format. The right format and the right answer aren't the same thing.

● KEY IDEA

Count every decision step's answers: an option never or always chosen is a warning, even without a crash.

The fix wasn't a better prompt or a better model. It was a design change. A threat-intel lookup takes a fraction of a second and costs nothing. Deciding *whether* to look it up costs about as much as just looking.

● KEY IDEA

When looking is cheaper than deciding whether to look, don't decide. Just look.

Comparing with the all-LLM agent

Now the comparison. The all-LLM agent runs the same loop, with the same tools and the same policy thresholds. The only difference is who decides. Every decision is an LLM call returning JSON, and the verdict is the LLM's own answer with its stated confidence. A broken answer goes to review, the fail-safe from Chapter 14.

Figure 17.4 shows the difference on a single alert, and it's mostly the width of the green, striped decide bars. Across the sample, the hybrid took 1.4 seconds per alert. The all-LLM agent took 6.2, about 4 times as long.

The bigger difference is the review queue (Figure 17.5). The all-LLM agent sent about 530 alerts a day to review, against a capacity of 240. You saw why in Chapter 13. Its stated confidences are grouped at a few round values, so a threshold at 0.03 has almost nothing to separate. Most alerts land between the thresholds. The hybrid sent 184.

A queue with more than twice its capacity doesn't get worked twice as hard. As Chapter 14 said, the extra alerts simply aren't looked at, and nobody chooses which ones. A threat can be missed in two ways: auto-closed, or never reached in the queue. Counting both, the hybrid let about 10 real threats a day go unseen by a person. The all-LLM agent let about 19 through.



Figure 17.4. One alert's trace as a timeline, for each agent. Plain blue steps observe, striped green steps decide, dotted orange steps write, cross-hatched grey steps act. The hybrid's decisions are very thin bars; the all-LLM agent's decisions take most of its time.

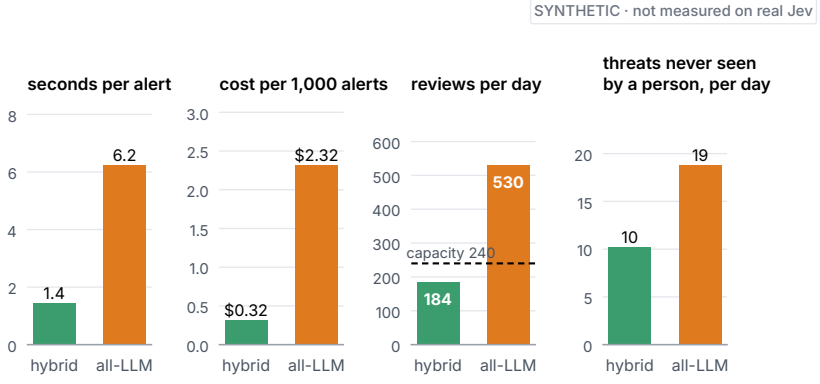


Figure 17.5. Four measures for the two agents on the same 700 live alerts, scaled to a day. The LLM's stated confidences sent far more alerts to review than the queue can take, so more real threats were never seen by a person.

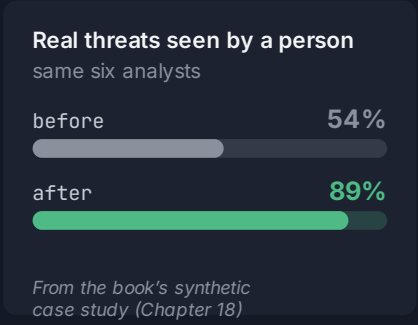
Most of what an AI agent does isn't writing. It's deciding.

Is this alert real? Which team gets this ticket? Is this action safe? Most agents give every one of those small decisions to a large language model. Then code has to find the answer inside a paragraph. It works, but it's slow and expensive, and the model sounds just as sure when it's wrong.

This book shows you a better way to build the decision layer. You'll get calibrated probabilities that mean what they say, thresholds set by what each mistake costs, and a clear rule for when to give a case to a person. Every step is built with runnable code, around a realistic (synthetic) security team. You'll work with Jev, a new System One model, through a free mock that needs no API key.

Inside, you'll learn to:

- Check whether a model's probabilities mean what they say, and fix them when they don't
- Turn a probability into act, review or escalate, with thresholds set by real costs and real capacity
- Compare six ways to make the same decision, from hand-written rules to LLMs to Jev
- Build a hybrid agent in which Jev decides and the LLM reads and writes
- Build your own small System One model, with typed heads and calibration built in



For students, engineers building agents, and tech leads deciding what to build.

About the author

Sridhar Mukkandi is an applied AI engineer who builds agents and the decision systems behind them. He writes for engineers who want AI systems they can trust.

sridhar.authorhub@gmail.com

An independent guide. Not affiliated with TypeSafe AI.

Labs, figures and code

Scan for the companion repository

Artificial Intelligence / Machine Learning

github.com/Mukkandi-Sridhar/decide-dont-generate